

INTEGRATIVE PROGRAMMING AND TECHNOLOGIES

Final Project

STOCKWISE: INVENTORY MANAGEMENT SYSTEM

A MERN Stack Web Application for Philippine Micro, Small and Medium Enterprises

Technical Documentation

Group Members

Ivan Tadena (Developer)

Course: Integrative Programming and Technologies

Section: INF238

Date: October 9, 2026

Live application: <https://stockwise-ivan.vercel.app>

Source code: <https://github.com/Ivan259483/stockwise>

Table of Contents

Project Overview	3
Objectives	4
Scope and Limitations	5
System Architecture	6
Database Design	9
User Interface Design	15
Feature List	25
Summary and Future Work	29
Sources	31

Project Overview

Background and Problem

Micro, small and medium enterprises (MSMEs) account for more than 99 percent of all business establishments in the Philippines, according to the Department of Trade and Industry. Many of them, such as neighborhood hardware stores, sari-sari stores and supply shops, still record their stock in paper logbooks or in loosely maintained spreadsheets.

Manual record keeping causes recurring problems. Owners discover that an item has run out only when a customer asks for it (stockouts), they reorder items that are already plentiful (overstocking), and items go missing without any record of when or by whom the quantity was changed. Because the records are scattered, the owner also cannot easily answer basic questions such as "How much is my current stock worth?" or "What do I need to reorder this week?"

Proposed Solution

StockWise is a web-based inventory management system built on the MERN stack (MongoDB, Express, React and Node.js). It replaces the logbook with a single, searchable product catalog and records every stock change as a *stock movement* that stores who made the change, when, why, and the quantity before and after it. A dashboard summarizes the total stock value in Philippine pesos, highlights products that are low or out of stock, and charts stock movements over the last seven days.

The system has two roles. An **admin** (typically the store owner) manages products, categories, suppliers and user accounts. A **staff** member can view all inventory data and record stock in and stock out, but cannot change the catalog. StockWise runs in any modern browser on a desktop computer or a mobile phone and is deployed publicly on Vercel with a MongoDB Atlas database.

Live Deliverables

Table 1. *Deployed resources*

Resource	URL
Web application	https://stockwise-ivan.vercel.app
REST API	https://stockwise-api-ivan.vercel.app
Source code	https://github.com/Ivan259483/stockwise
Demo admin account	admin@stockwise.com / Admin123!
Demo staff account	staff@stockwise.com / Staff123!

Objectives

The general objective of the project is to develop a secure, responsive web application that allows a small Philippine retail business to track its inventory accurately and to know what to reorder. The specific, measurable objectives are the following:

- **Secure access:** require a valid JSON Web Token (JWT) for every endpoint except health, login and registration, and enforce two roles (admin and staff) on both the server and the user interface.
- **Complete data management:** provide create, read, update and delete (CRUD) operations for products, categories and suppliers, and user management for administrators, all through a RESTful API.
- **Full audit trail:** record every change to a product's stock quantity as a stock movement that stores the previous quantity, the new quantity, the reason and the user who made the change.
- **No negative stock:** reject every stock-out request that exceeds the available quantity with HTTP status 409, using an atomic database update so that simultaneous requests cannot push stock below zero.
- **Low-stock alerts:** mark a product as "Low stock" when its quantity is at or below its reorder level and as "Out of stock" at zero, and list the five most urgent products on the dashboard.

- **Reliable input:** validate every write request on the server with express-validator and every form on the client before submission, and return errors in one consistent format.
- **Responsive design:** keep every page usable without horizontal scrolling from a 360-pixel-wide phone screen up to a desktop monitor.
- **Verified quality:** pass an automated 40-check API smoke test and a 30-step browser end-to-end test against both the local and the deployed system.

Scope and Limitations

Scope

- User registration and login with JWT authentication, bcrypt password hashing and two roles (admin and staff).
- Management of products, categories and suppliers, including an optional product image of up to 1 MB.
- Stock-in and stock-out recording with reasons (for example Purchase, Sale, Damaged or Expired) and a filterable movement history.
- A dashboard with summary statistics, a stock-value-by-category bar chart, a seven-day stock movement line chart, a restock list and recent movements.
- User administration: changing roles, activating or deactivating accounts, and deleting accounts.
- Input validation and error handling on both the client and the server.
- A responsive interface for desktop, tablet and mobile browsers.

Limitations

- The system manages a single store. It does not support multiple branches or transfers of stock between locations.
- It is not a point-of-sale (POS) system. Sales are recorded as stock-out movements, without receipts, payments or customer records.
- Products are identified by SKU text only; barcode and QR code scanning are not supported.
- Quantities are whole numbers. Items sold by weight or volume must be tracked in whole units such as kilograms or liters.

- Low-stock alerts appear inside the application only; no email or SMS notifications are sent.
- Product images are stored inside the database as data URLs and are limited to 1 MB each.
- There is no self-service password reset; an administrator must manage accounts.
- The free hosting tiers may add a short delay to the first request after a period of inactivity (a serverless "cold start"), and the login rate limit is kept in memory per serverless instance.

System Architecture

StockWise follows a three-tier architecture. The presentation tier (React) and the application tier (Express) are deployed as two separate Vercel projects and communicate only through a RESTful JSON API over HTTPS. The data tier is a MongoDB database hosted on MongoDB Atlas. Figure 1 shows the tiers and the path of a request through the API.

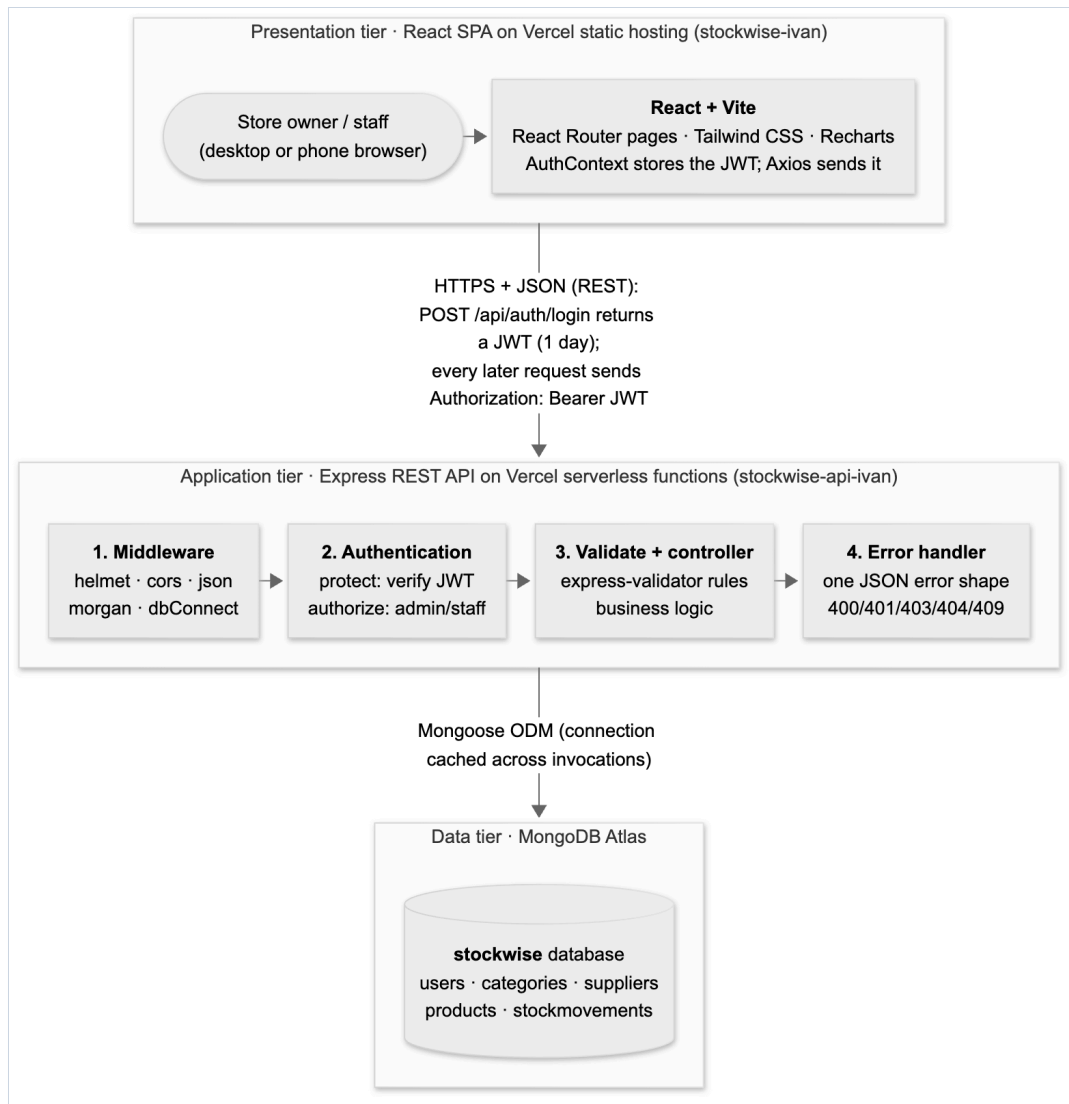


Figure 1. *System architecture of StockWise*

Presentation Tier (Client)

The client is a single-page application built with React 19 and Vite and styled with Tailwind CSS. React Router maps each URL to a page, and route guards (ProtectedRoute and AdminRoute) keep signed-out users and staff away from pages they may not use. All server calls go through one Axios instance whose interceptors attach the JWT to every request, show an error notification for every failed request, and end the session automatically when the server answers 401 (Unauthorized). The production build is served as static files by Vercel, and a rewrite rule sends every path to index.html so that deep links such as /products/123 work when the page is refreshed.

Application Tier (REST API)

The API is an Express 5 application written in Node.js with ES modules. It is organized by responsibility into models, controllers, routes, middleware, validators and utilities. Every request passes through the same pipeline:

1. **Security and parsing middleware:** helmet sets secure HTTP headers, cors allows only the deployed client and localhost, express.json parses the body (up to 5 MB so that images fit), and morgan logs the request.
2. **Database connection:** the dbConnect middleware opens the MongoDB connection or reuses the one cached on the serverless instance.
3. **Authentication and authorization:** protect verifies the JWT and reloads the user from the database, so a deactivated user is blocked immediately; authorize then checks the user's role.
4. **Validation and controller:** express-validator rules check the request, and the controller performs the business logic using Mongoose models.
5. **Central error handler:** every error is converted into the same JSON shape with an appropriate HTTP status code.

On Vercel, the Express application is exported from server.js and runs as a serverless function, while app.listen is called only during local development. The Mongoose connection promise is cached across invocations to avoid opening a new connection pool for every request.

Data Tier (Database)

MongoDB Atlas stores five collections in a database named stockwise. Mongoose schemas define the structure, validation rules, indexes and relationships of the documents, as described in the Database Design section.

REST and JWT Authentication Flow

1. The user submits the login form. The client sends POST /api/auth/login with the email address and password.
2. The API finds the user, compares the password with the stored bcrypt hash and, if they match, returns a JWT signed with a secret key that expires after one day, together with the user's profile.

3. The client stores the token in localStorage, and the Axios interceptor adds the header "Authorization: Bearer <token>" to every later request.
4. For each protected request, the API verifies the token, loads the user and checks the role. It responds with JSON in the form { success: true, data } on success or { success: false, message, errors } on failure.
5. If the token is missing, invalid or expired, the API returns 401; the client clears the session and redirects to the login page.

Stock Integrity

A stock-out is performed with a single atomic MongoDB operation that decreases the quantity only if enough stock is available (a findOneAndUpdate with the condition $\text{quantity} \geq \text{requested amount}$). If the condition fails, the API returns 409 (Conflict) with the message "Insufficient stock: only X available". Because the check and the update happen in one database operation, two users recording sales at the same moment can never make the stock negative. Each successful change then writes a stock movement record; if writing the record fails, the quantity change is reversed so that stock levels never change without a matching history entry.

Database Design

Figures 2 and 3 show the five collections and their relationships. Figure 2 covers the product catalog: products reference their category and, optionally, their supplier. Figure 3 covers the stock history: stock movements reference the product and the user who recorded them, and they also store a copy of the product name and SKU so that the history remains readable after a product is renamed or deleted (Figure 3 repeats only the key fields of products). The constraints of every field are listed in Tables 2 to 6. All collections include automatic createdAt and updatedAt timestamps, which are omitted from the diagrams for clarity.

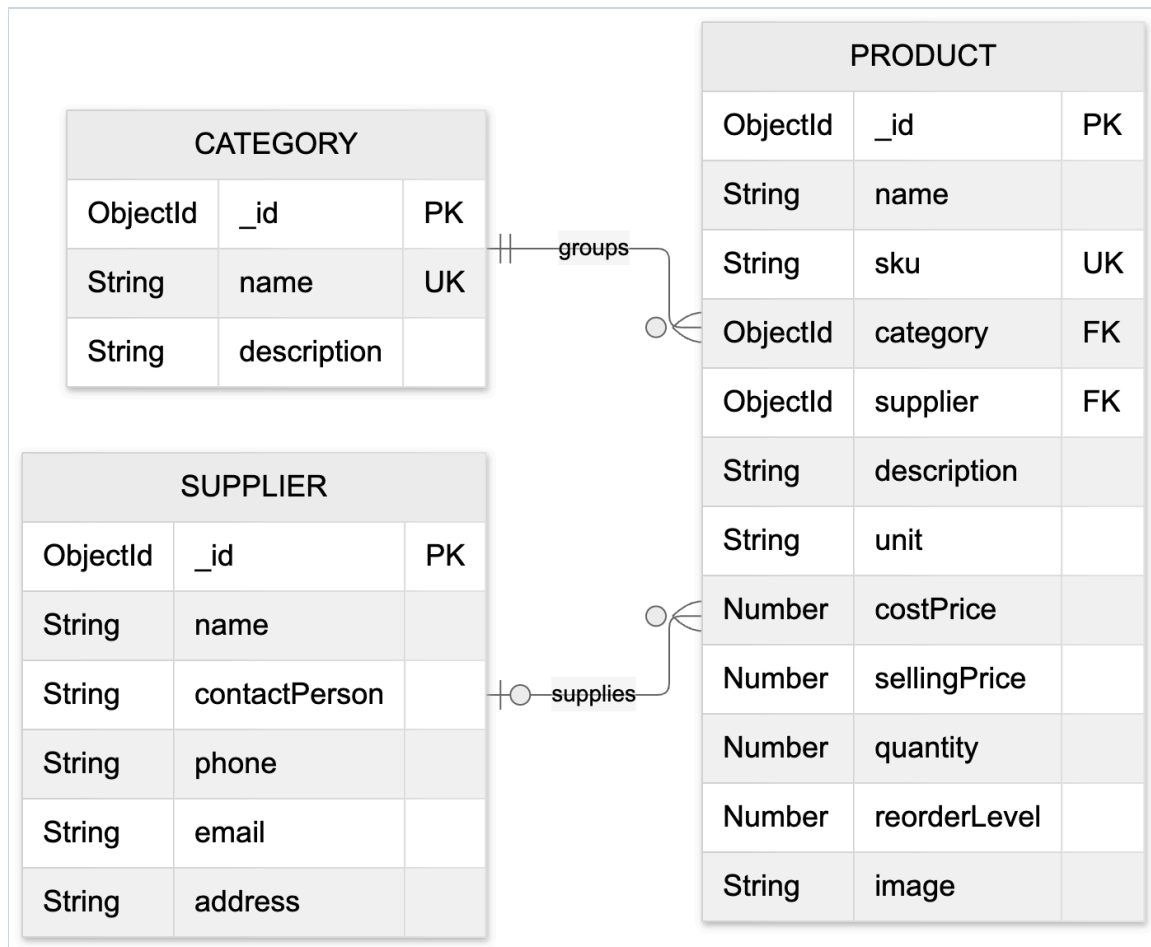


Figure 2. Entity relationship diagram, part 1: product catalog

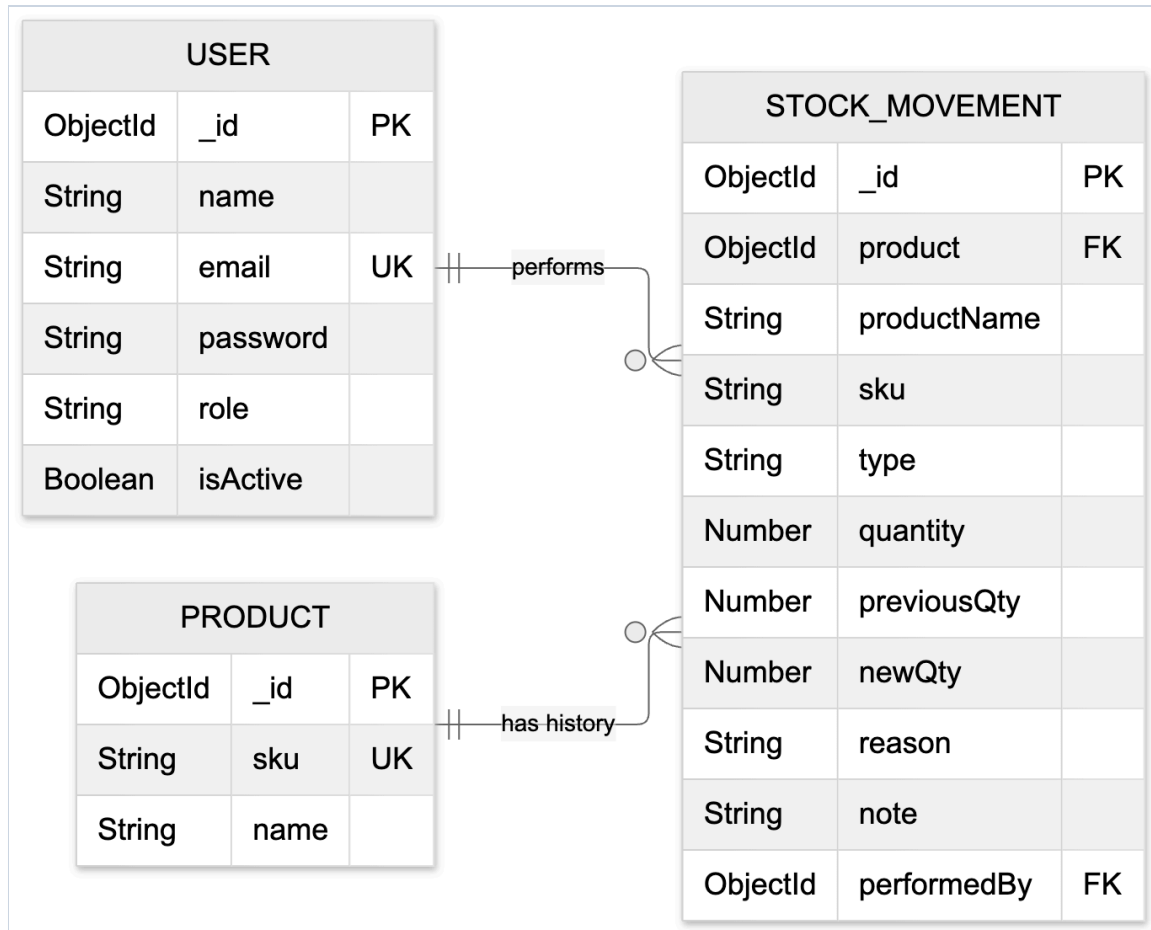


Figure 3. Entity relationship diagram, part 2: stock history

- **Category to Product (one to many):** a category groups many products; every product must belong to exactly one category. A category that is still in use cannot be deleted.
- **Supplier to Product (zero or one to many):** a supplier may supply many products; the supplier is optional. Deleting a supplier removes the reference from its products.
- **Product to StockMovement (one to many):** each product has a history of stock movements.
- **User to StockMovement (one to many):** each movement records the user who performed it.

Table 2. users collection

Field	Type	Constraints	Description
_id	ObjectId	Primary key	Unique identifier
name	String	Required, max 60 characters	Full name

Field	Type	Constraints	Description
email	String	Required, unique, lowercase, valid format	Login email address
password	String	Required, min 8 characters, hidden from queries	bcrypt hash of the password
role	String	admin or staff, default staff	Determines permissions
isActive	Boolean	Default true	Inactive users cannot log in
createdAt, updatedAt	Date	Automatic	Timestamps

Table 3. *categories collection*

Field	Type	Constraints	Description
_id	ObjectId	Primary key	Unique identifier
name	String	Required, unique (case-insensitive), max 50	Category name, e.g. Plumbing
description	String	Optional, max 300 characters	Short description
createdAt, updatedAt	Date	Automatic	Timestamps

Table 4. *suppliers collection*

Field	Type	Constraints	Description
_id	ObjectId	Primary key	Unique identifier
name	String	Required, max 80 characters	Supplier business name
contactPerson	String	Optional	Name of the contact person
phone	String	Optional, valid phone format	Contact number
email	String	Optional, valid email format	Contact email address
address	String	Optional, max 200 characters	Business address

Field	Type	Constraints	Description
createdAt, updatedAt	Date	Automatic	Timestamps

Table 5. *products collection*

Field	Type	Constraints	Description
_id	ObjectId	Primary key	Unique identifier
name	String	Required, max 100 characters; text index	Product name
sku	String	Required, unique, uppercase; text index	Stock keeping unit code
category	ObjectId	Required, references categories	Product category
supplier	ObjectId	Optional, references suppliers	Usual supplier
description	String	Optional, max 500 characters	Product details
unit	String	pcs, box, kg, L or pack; default pcs	Unit of measure
costPrice	Number	≥ 0	Purchase cost per unit (₱)
sellingPrice	Number	≥ 0	Selling price per unit (₱)
quantity	Number	Integer ≥ 0 , default 0	Units currently in stock
reorderLevel	Number	Integer ≥ 0 , default 10	Low-stock threshold
image	String	Optional URL or image data URL ≤ 1 MB	Product photo
stockStatus	Virtual	Computed, not stored	in_stock, low_stock or out_of_stock
stockValue	Virtual	Computed, not stored	quantity $\times$ costPrice
createdAt, updatedAt	Date	Automatic	Timestamps

Table 6. *stockmovements collection*

Field	Type	Constraints	Description
_id	ObjectId	Primary key	Unique identifier
product	ObjectId	Required, references products	Product that changed
productName, sku	String	Required	Snapshot kept after deletion
type	String	IN or OUT	Direction of the movement
quantity	Number	Integer > 0	Units moved
previousQty, newQty	Number	Required	Stock before and after
reason	String	Purchase, Return, Sale, Damaged, Expired, Adjustment or Other	Why the stock changed
note	String	Optional, max 300 characters	Free-text remark
performedBy	ObjectId	Required, references users	User who recorded it
createdAt, updatedAt	Date	Automatic; indexed	When it happened

User Interface Design

The interface uses a consistent indigo and slate color palette, rounded cards and clear spacing. Color-coded badges show stock status (green for in stock, amber for low stock and red for out of stock). All screenshots below were captured from the live deployment at a desktop width of 1440 pixels and a mobile width of 390 pixels.

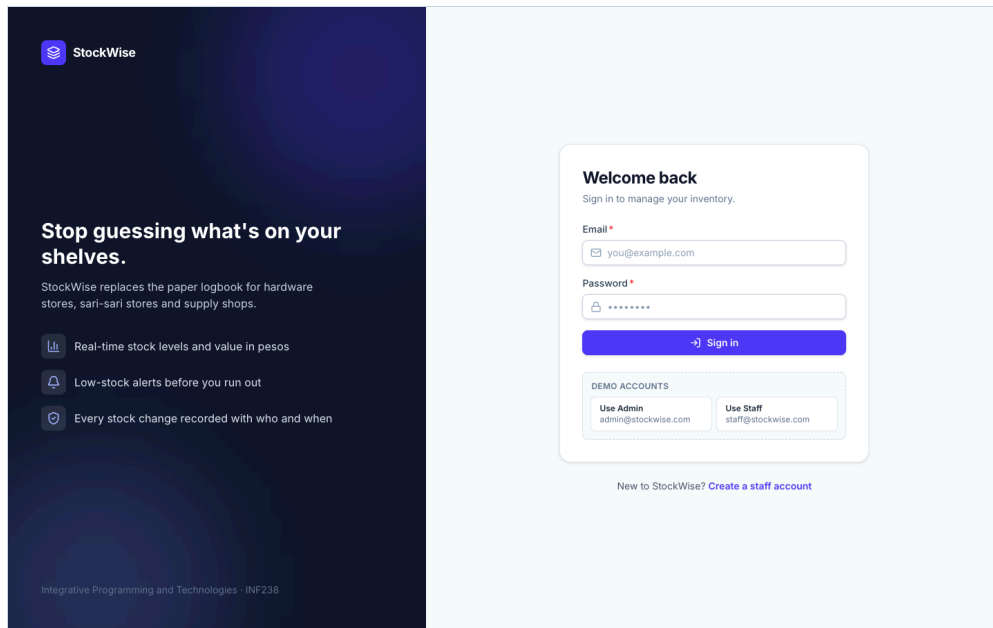
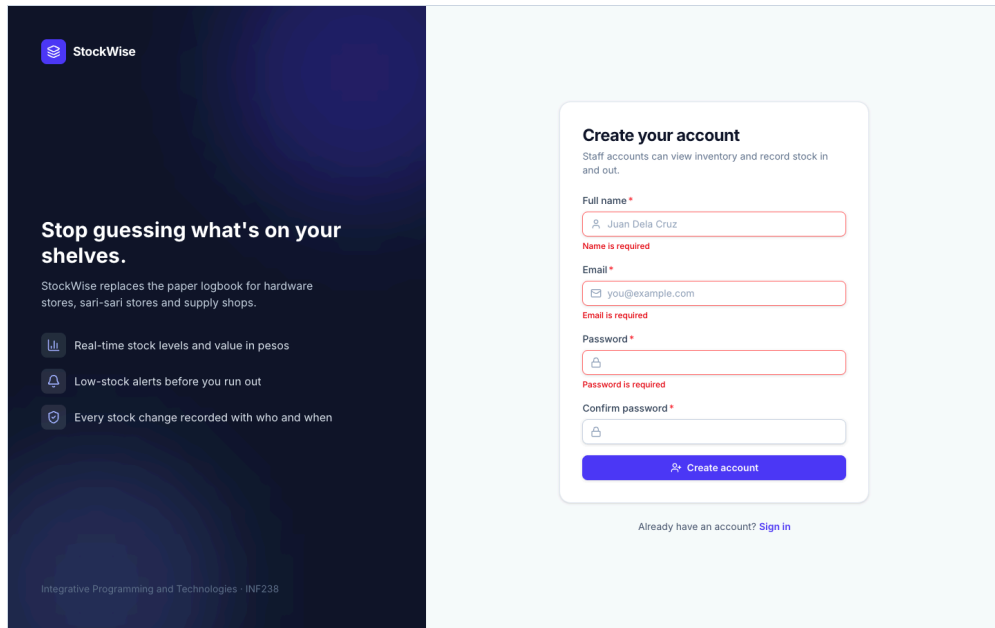


Figure 4. *Login page*

The login page has a centered form with inline validation. Demo-account buttons fill in the admin or staff credentials so reviewers can try both roles quickly.



StockWise

Stop guessing what's on your shelves.

StockWise replaces the paper logbook for hardware stores, sari-sari stores and supply shops.

- Real-time stock levels and value in pesos
- Low-stock alerts before you run out
- Every stock change recorded with who and when

Integrative Programming and Technologies - INF238

Create your account

Staff accounts can view inventory and record stock in and out.

Full name *

Name is required

Email *

Email is required

Password *

Password is required

Confirm password *

Create account

Already have an account? [Sign in](#)

Figure 5. Registration form with validation errors

Public registration creates staff accounts. Each invalid field shows its own message before any request is sent to the server.

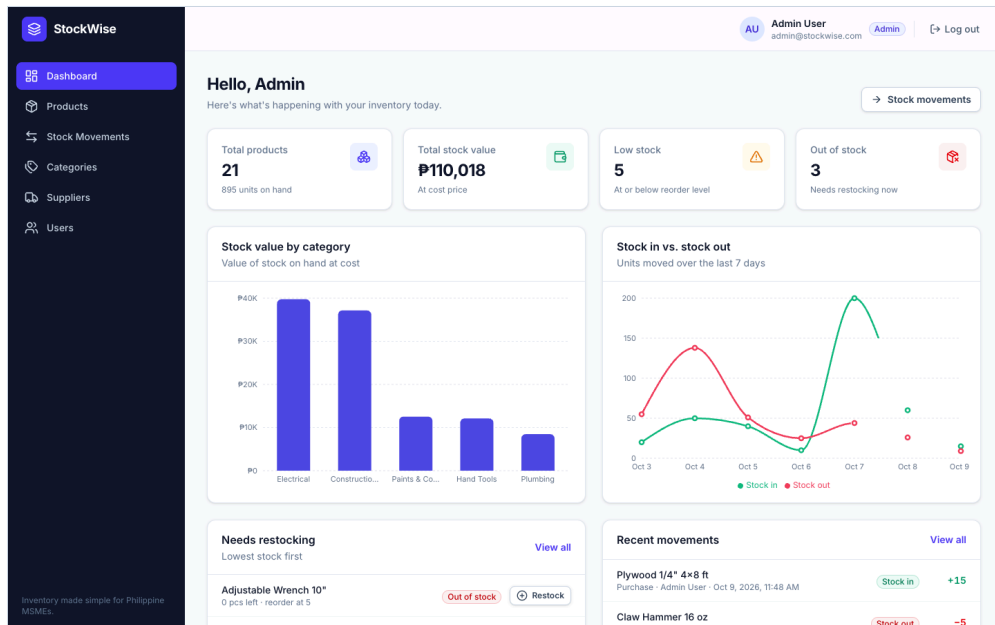


Figure 6. Admin dashboard

The dashboard shows the total number of products, the total stock value in pesos, and the low-stock and out-of-stock counts. Below them are a bar chart of stock value by category and a line

chart comparing units stocked in and out over the last seven days.

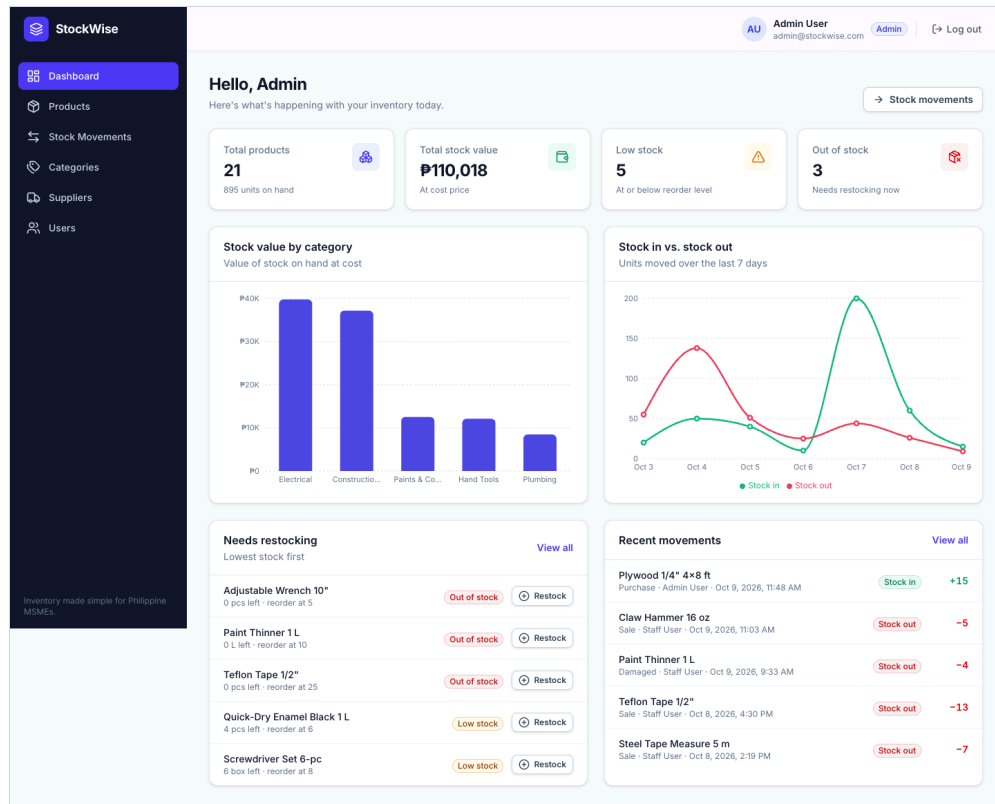


Figure 7. Dashboard restock list and recent movements

The "Needs restocking" list shows the most urgent products, each with a Restock button that opens the stock-in form for that product. The recent movements panel shows the latest stock changes and who made them.

PRODUCT	CATEGORY	QTY	PRICE	STOCK VALUE	STATUS	ACTIONS
Adjustable Wrench 10" HT-WRI-010	Hand Tools	0 pcs	₱340.00	₱0.00	Out of stock	👁️ ✎️ 🗑️
Brass Faucet 1/2" PL-FAU-050	Plumbing	18 pcs	₱265.00	₱4,770.00	In stock	👁️ ✎️ 🗑️
Claw Hammer 16 oz HT-HAM-015	Hand Tools	34 pcs	₱260.00	₱8,840.00	In stock	👁️ ✎️ 🗑️
Common Nails 2" (per kg) CH-NAI-002	Construction Materials	28 kg	₱105.00	₱2,940.00	In stock	👁️ ✎️ 🗑️
Concrete Hollow Block 4" CH-CHB-004	Construction Materials	350 pcs	₱16.00	₱5,600.00	In stock	👁️ ✎️ 🗑️
Convenience Outlet Duplex EL-OUT-002	Electrical	9 pcs	₱75.00	₱675.00	Low stock	👁️ ✎️ 🗑️
Deformed Bar 10 mm x 6 m CH-RBR-010	Construction Materials	8 pcs	₱245.00	₱1,960.00	Low stock	👁️ ✎️ 🗑️
Extension Cord 4-Gang 3 m EL-EXT-004	Electrical	22 pcs	₱380.00	₱8,360.00	In stock	👁️ ✎️ 🗑️
LED Bulb 9W Daylight EL-LED-009	Electrical	120 pcs	₱99.00	₱11,880.00	In stock	👁️ ✎️ 🗑️
Latex Paint White 4 L PT-LAT-004	Paints & Coatings	16 pcs	₱795.00	₱12,720.00	In stock	👁️ ✎️ 🗑️

Figure 8. Products list (desktop)

The products page supports search by name or SKU (debounced by 300 ms), filtering by category and stock status, sorting and pagination. Administrators see view, edit and delete actions for each product.

PRODUCT	CATEGORY	QTY	PRICE	STOCK VALUE	STATUS	ACTIONS
Convenience Outlet Duplex EL-OUT-002	Electrical	9 pcs	₱75.00	₱675.00	Low stock	👁️ ✎️ 🗑️
Deformed Bar 10 mm x 6 m CH-RBR-010	Construction Materials	8 pcs	₱245.00	₱1,960.00	Low stock	👁️ ✎️ 🗑️
PVC Elbow 1/2" 90° PL-ELB-050	Plumbing	14 pcs	₱15.00	₱210.00	Low stock	👁️ ✎️ 🗑️
Quick-Dry Enamel Black 1 L PT-QDE-001	Paints & Coatings	4 pcs	₱345.00	₱1,380.00	Low stock	👁️ ✎️ 🗑️
Screwdriver Set 6-pc HT-SCR-006	Hand Tools	6 box	₱320.00	₱1,920.00	Low stock	👁️ ✎️ 🗑️

1-5 of 5 Page 1 of 1

Figure 9. Products filtered by low stock

Filters are stored in the page address, so the dashboard's "Low stock" card links directly to this filtered view.

StockWise

Admin ^{AU} admin@ Please fix the highlighted fields.

← Back

Add product

Add a new item to your catalogue.

Product details

Product name *

Product name is required

SKU *

Unit

Use 2-30 letters, numbers or dashes (e.g. HT-HAM-016)

Category *

Supplier

Please choose a category

Description

Image

Optional. PNG, JPG, GIF or WebP up to 1 MB.

Upload image

Create product

Cancel

Pricing and stock

Cost price (P) *

Enter a cost price of 0 or more

Selling price (P) *

Enter a selling price of 0 or more

Figure 10. *Product form with validation errors*

The product form validates every field before submission and also shows field errors returned by the server, such as a duplicate SKU.

StockWise

Admin User admin@stockwise.com Admin Log out

← Products

Claw Hammer 16 oz

SKU HT-HAM-016

Stock in Stock out Edit

Product information

	CATEGORY Hand Tools	SUPPLIER Manila Hardware Trading Co.	UNIT pcs
	COST PRICE P185.00	SELLING PRICE P260.00	ADDED Oct 9, 2026

Claw Hammer 16 oz. Stocked by Manila Hardware Trading Co..

Supplier contact: Ramon Dela Cruz - 0917 555 0142

Current stock

34 pcs

In stock

Reorder level 10

Stock value (cost) P6,290.00

Potential sales P8,840.00

Movement history

Last 20 stock movements for this product

[View full history](#)

DATE	TYPE	QTY	STOCK	REASON	NOTE	BY
Oct 9, 2026, 11:03 AM	Stock out	-5	39 → 34	Sale	—	Staff User
Oct 3, 2026, 9:15 AM	Stock in	+20	19 → 39	Purchase	PO #1021 from Manila Hardware	Admin User

Figure 11. *Product details with movement history*

The product details page shows the product information, current stock, status, stock value and potential sales, followed by the product's last 20 stock movements.

The screenshot shows the 'Record stock movement' modal for the product 'Claw Hammer 16 oz'. The modal is open, displaying the current stock level of 34 pcs. The user has selected 'Stock in' as the movement type, entered a quantity of 12, and chosen 'Purchase' as the reason. A note is entered: 'PO #1026 from Manila Hardware'. The 'Record stock in' button is highlighted in green.

Figure 12. *Stock movement form*

Stock in and stock out are recorded in a modal dialog with the movement type, quantity, reason and an optional note. The current stock is shown for reference.

The screenshot shows the 'Record stock movement' modal for the product 'Screwdriver Set 6-pc'. The modal is open, displaying the current stock level of 6 boxes. The user has selected 'Stock out' as the movement type, entered a quantity of 50, and chosen 'Sale' as the reason. A red error message 'Insufficient stock: only 6 available' is displayed above the 'Record stock out' button. The 'Record stock out' button is highlighted in red.

Figure 13. *Insufficient stock error*

When a stock-out is larger than the available quantity, the API rejects it with status 409. The message appears both as a notification and under the quantity field.

Stock movements
31 movements recorded

Product: All products Type: All types From: mm/dd/yyyy To: mm/dd/yyyy Clear

DATE	PRODUCT	TYPE	QTY	STOCK	REASON	NOTE	BY
Oct 9, 2026, 11:48 AM	Plywood 1/4" 4x8 ft CY-PLY-025	Stock in	+15	11 → 26	Purchase	PO #1025	Admin User
Oct 9, 2026, 11:03 AM	Claw Hammer 16 oz HT-HAM-016	Stock out	-5	39 → 34	Sale	—	Staff User
Oct 9, 2026, 9:33 AM	Paint Thinner 1 L PT-TNN-001	Stock out	-4	4 → 0	Damaged	Leaking containers	Staff User
Oct 8, 2026, 4:30 PM	Teflon Tape 1/2" PL-TEF-036	Stock out	-13	13 → 0	Sale	Sold out	Staff User
Oct 8, 2026, 2:19 PM	Steel Tape Measure 5 m HT-TAP-005	Stock out	-7	55 → 48	Sale	—	Staff User
Oct 8, 2026, 11:45 AM	Latex Paint White 4 L PT-LAT-004	Stock out	-6	22 → 16	Sale	—	Staff User
Oct 8, 2026, 9:10 AM	LED Bulb 9W Daylight EL-LED-009	Stock in	+60	60 → 120	Purchase	PO #1024	Admin User
Oct 7, 2026, 5:10 PM	Paint Brush 2" PT-BRS-002	Stock out	-10	55 → 45	Sale	—	Staff User
Oct 7, 2026, 3:40 PM	Extension Cord 4-Gang 3 m EL-EXT-004	Stock out	-5	27 → 22	Sale	—	Staff User
Oct 7, 2026, 1:30 PM	PVC Elbow 1/2" 90° PL-ELB-009	Stock out	-26	40 → 14	Sale	Plumbing contractor	Staff User

Figure 14. Stock movements history

The stock movements page is the complete audit trail. It can be filtered by product, type and date range, and new movements can be recorded from here.

Categories
Group your products so they are easier to find and report on.

+ Add category

NAME	DESCRIPTION	PRODUCTS	ACTIONS
Construction Materials	Cement, rebar, plywood, nails and hollow blocks	5 products	
Electrical	Wires, outlets, bulbs and extension cords	4 products	
Hand Tools	Hammers, wrenches, screwdrivers and measuring tools	4 products	
Paints & Coatings	Latex and enamel paints, thinners and brushes	4 products	
Plumbing	PVC pipes, fittings, faucets and sealants	4 products	

Figure 15. Categories page

Categories are listed with the number of products in each. Administrators create and edit categories in a modal form; deleting a category that still has products is refused.

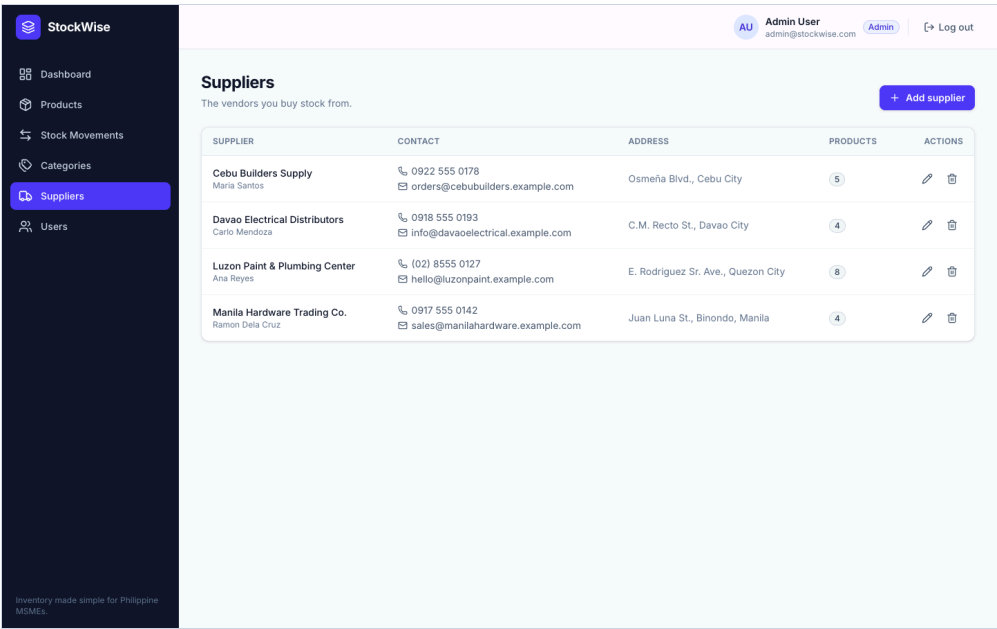


Figure 16. Suppliers page

The supplier directory shows contact persons, phone numbers, email addresses and the number of products each supplier provides.

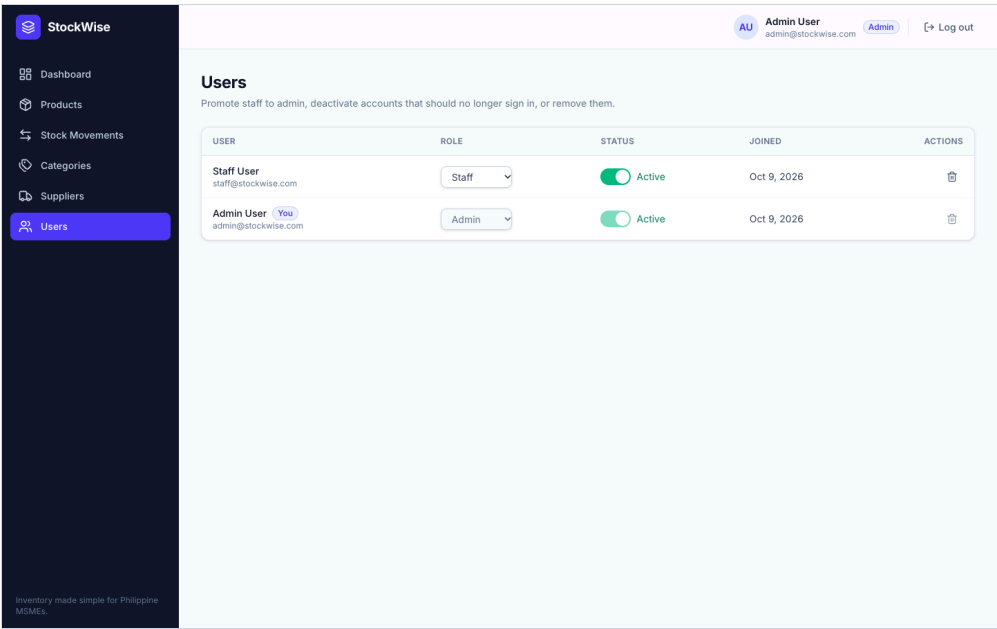


Figure 17. User management (admin only)

Administrators can change roles, activate or deactivate accounts and delete users. The controls for the administrator's own account are disabled to prevent accidental lock-out.

PRODUCT	CATEGORY	QTY	PRICE	STOCK VALUE	STATUS	ACTIONS
Adjustable Wrench 10" HT-MRW-010	Hand Tools	0 pcs	P340.00	P0.00	Out of stock	🔍
Brass Faucet 1/2" PL-FAU-050	Plumbing	18 pcs	P265.00	P3,240.00	In stock	🔍
Claw Hammer 16 oz HT-HAW-010	Hand Tools	34 pcs	P260.00	P6,290.00	In stock	🔍
Common Nails 2" (per kg) CH-NAL-002	Construction Materials	28 kg	P105.00	P2,100.00	In stock	🔍
Concrete Hollow Block 4" CH-CHB-004	Construction Materials	350 pcs	P16.00	P4,200.00	In stock	🔍
Convenience Outlet Duplex EL-OUT-002	Electrical	9 pcs	P75.00	P432.00	Low stock	🔍
Deformed Bar 10 mm x 6 m CH-RBR-010	Construction Materials	8 pcs	P245.00	P1,560.00	Low stock	🔍
Extension Cord 4-Gang 3 m EL-EXT-004	Electrical	22 pcs	P380.00	P5,720.00	In stock	🔍
LED Bulb 9W Daylight EL-LED-009	Electrical	120 pcs	P99.00	P7,800.00	In stock	🔍
Latex Paint White 4 L PT-LAT-004	Paints & Coatings	16 pcs	P795.00	P9,920.00	In stock	🔍

Figure 18. *Products page as seen by staff*

Staff users see the same inventory data, but the Users link and all product editing actions are hidden. The API enforces the same restrictions.

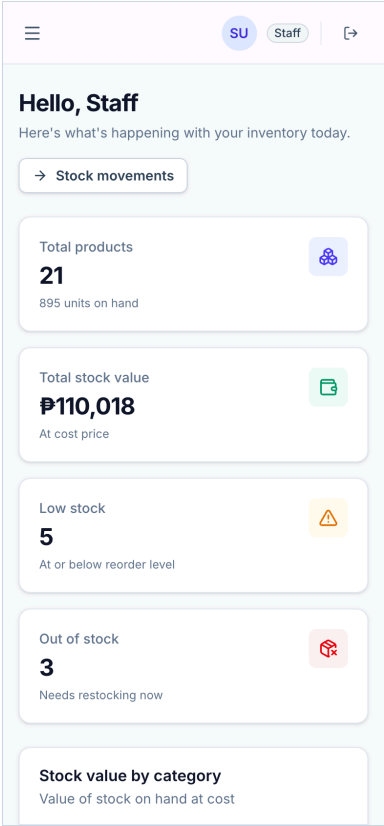


Figure 19. Dashboard on a phone

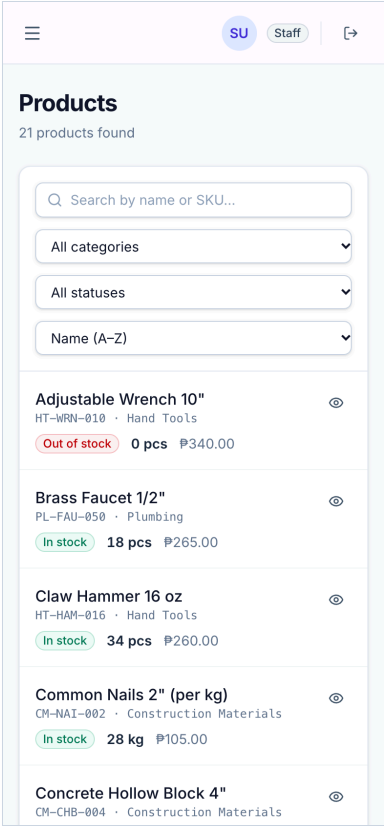


Figure 20. Products as mobile cards

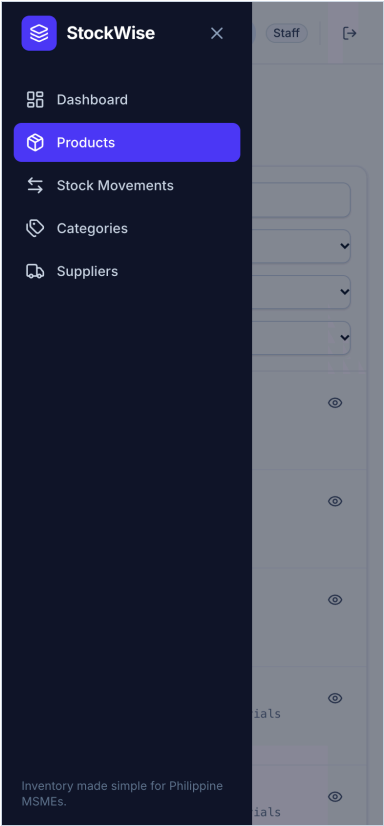


Figure 21. Navigation drawer

On small screens the sidebar becomes a drawer opened from the menu button, stat cards stack vertically, and tables are replaced by easy-to-read cards.

Feature List

Authentication and Roles

- Registration of staff accounts and login with JSON Web Tokens that expire after one day.
- Passwords hashed with bcrypt and never returned by the API.
- Rate limiting of failed login and registration attempts (20 per 15 minutes per IP address).
- Two roles: admins manage everything; staff view inventory and record stock movements.
- Protected and admin-only routes in the client; automatic logout when the session expires.

Dashboard

- Statistic cards: total products and units, total stock value in pesos, low-stock count and out-of-stock count.
- Bar chart of stock value by category and line chart of units stocked in and out over the last seven days.
- Restock list of the five most urgent products with a one-click Restock action.
- List of the five most recent stock movements.

Products

- Search by name or SKU with a 300 ms debounce, filters by category and stock status, sorting and pagination.
- Table layout on desktop and card layout on mobile, with color-coded status badges.
- Create, edit and delete products (admin), with category and supplier selection and an optional image of up to 1 MB with preview.
- Opening stock recorded automatically as the first stock movement.
- Product details page with stock value, potential sales and the last 20 movements.

Categories

- List of categories with the number of products in each.
- Create and edit categories in a modal form (admin); names are unique regardless of letter case.

- Deletion is blocked with a clear message while products still use the category.

Suppliers

- Supplier directory with contact person, phone, email, address and product count.
- Create, edit and delete suppliers (admin); deleting a supplier removes it from its products.

Stock Movements

- Stock in (Purchase, Return, Adjustment, Other) and stock out (Sale, Damaged, Expired, Adjustment, Other) for both roles.
- Atomic updates that never allow negative stock; oversized stock-outs return "Insufficient stock: only X available".
- Complete history with previous and new quantities, filterable by product, type and date range, with pagination.

Users

- List of all accounts with role selection, an active/inactive switch and deletion (admin only).
- Administrators cannot demote, deactivate or delete their own account.

Validation and Error Handling

- express-validator rules on every write endpoint and matching client-side validation on every form.
- One central error handler that returns { success: false, message, errors } with field-level details.
- Correct status codes: 400 for invalid input or IDs, 401 for missing or invalid tokens, 403 for forbidden roles, 404 for unknown resources or routes, 409 for duplicates and insufficient stock, and 500 for unexpected errors, whose details are logged only on the server.
- Inline field errors, a notification for every failed request, and loading states that disable buttons while a request is in progress.

Responsive Design

- No horizontal scrolling on any page, verified at widths of 360, 375 and 390 pixels and on desktop screens.

- Collapsible navigation drawer, stacked cards on mobile, and accessible labels on all form controls.
- Loading, empty and error states on every list.

API Endpoints

Table 7. *REST API endpoints (base path /api)*

Method	Endpoint	Access	Purpose
GET	/api/health	Public	Health check
POST	/api/auth/register	Public	Create a staff account
POST	/api/auth/login	Public	Log in and receive a JWT
GET	/api/auth/me	Logged in	Current user
GET	/api/users	Admin	List users
PATCH	/api/users/:id	Admin	Change role or active status
DELETE	/api/users/:id	Admin	Delete a user
GET	/api/categories	Logged in	List categories with product counts
POST	/api/categories	Admin	Create a category
PUT	/api/categories/:id	Admin	Update a category
DELETE	/api/categories/:id	Admin	Delete an unused category
GET	/api/suppliers	Logged in	List suppliers
POST	/api/suppliers	Admin	Create a supplier
PUT	/api/suppliers/:id	Admin	Update a supplier
DELETE	/api/suppliers/:id	Admin	Delete a supplier
GET	/api/products	Logged in	Search, filter, sort and paginate products
GET	/api/products/:id	Logged in	Product details and last 20 movements
POST	/api/products	Admin	Create a product

Method	Endpoint	Access	Purpose
PUT	/api/products/:id	Admin	Update a product
DELETE	/api/products/:id	Admin	Delete a product
POST	/api/stock/in	Logged in	Record a stock-in movement
POST	/api/stock/out	Logged in	Record a stock-out movement
GET	/api/stock/movements	Logged in	Filtered movement history
GET	/api/dashboard/summary	Logged in	Dashboard statistics and charts

Summary and Future Work

Summary

StockWise demonstrates how the MERN stack can solve a common problem of Philippine MSMEs: inaccurate, untraceable inventory records. The application integrates a React client with an Express REST API and a MongoDB Atlas database, and it covers the core concepts of the course, namely authentication, CRUD operations, database management, API integration, input validation, error handling and responsive user interface design.

The code is modular and documented. The server separates models, controllers, routes, middleware and validators, and the client is built from reusable components and custom hooks. ESLint and Prettier enforce consistent coding standards.

Testing and Verification

- **API smoke test:** 40 automated checks covering login, CRUD for products, categories and suppliers, stock in and out, insufficient stock (409), invalid IDs (400), staff access to admin routes (403), missing tokens (401) and duplicate SKUs (409). All 40 checks passed against both the local and the deployed API.
- **Browser end-to-end test:** 30 automated steps in headless Chrome for both roles, covering validation errors, product CRUD, stock movements, the insufficient-stock error, the 375-pixel mobile layout, refreshing a nested route and session expiry. All 30 steps passed on the live site.
- **Code quality:** the client builds without warnings and both projects pass ESLint with no errors.

Future Work

- **Barcode and QR code scanning** with the phone camera to find products and record movements faster.
- **Purchase orders** that are created from the restock list and update stock automatically when the goods arrive.
- **Multi-branch support** with separate stock levels per branch and transfers between branches.

- **Sales and POS integration** so that each sale automatically records a stock-out with receipt details.
- **CSV and PDF reports** of inventory valuation, movement history and low-stock items.
- **Email low-stock alerts** sent to the owner when a product reaches its reorder level.

Sources

Frameworks, Libraries and Services

1. Node.js. (n.d.). *Node.js documentation*. <https://nodejs.org/>
2. Express. (n.d.). *Express: Fast, unopinionated, minimalist web framework for Node.js*. <https://expressjs.com/>
3. MongoDB, Inc. (n.d.). *MongoDB Atlas*. <https://www.mongodb.com/atlas>
4. Mongoose. (n.d.). *Mongoose ODM documentation*. <https://mongoosejs.com/>
5. Meta Open Source. (n.d.). *React*. <https://react.dev/>
6. Vite. (n.d.). *Vite: Next generation frontend tooling*. <https://vite.dev/>
7. Tailwind Labs. (n.d.). *Tailwind CSS*. <https://tailwindcss.com/>
8. Remix Software. (n.d.). *React Router*. <https://reactrouter.com/>
9. Axios. (n.d.). *Axios: Promise based HTTP client*. <https://axios-http.com/>
10. Recharts. (n.d.). *Recharts: A composable charting library built on React components*. <https://recharts.org/>
11. react-hot-toast. (n.d.). *Smoking hot React notifications*. <https://react-hot-toast.com/>
12. Lucide. (n.d.). *Lucide icons*. <https://lucide.dev/>
13. Auth0. (n.d.). *jsonwebtoken*. <https://github.com/auth0/node-jwebtoken>
14. bcrypt.js. (n.d.). *Optimized bcrypt in JavaScript*. <https://github.com/dcodeIO/bcrypt.js>
15. express-validator. (n.d.). *express-validator documentation*. <https://express-validator.github.io/>
16. Helmet. (n.d.). *Helmet: Secure Express apps with HTTP headers*. <https://helmetjs.github.io/>
17. Express. (n.d.). *cors middleware*. <https://github.com/expressjs/cors>
18. Express. (n.d.). *morgan HTTP request logger*. <https://github.com/expressjs/morgan>
19. express-rate-limit. (n.d.). *Basic rate-limiting middleware for Express*. <https://github.com/express-rate-limit/express-rate-limit>
20. dotenv. (n.d.). *Loads environment variables from a .env file*. <https://github.com/motdotla/dotenv>
21. Vercel. (n.d.). *Express on Vercel*. <https://vercel.com/docs/frameworks/backend/express>

22. Vercel. (n.d.). *Configuring projects with vercel.json*. <https://vercel.com/docs/project-configuration/vercel-json>
23. Puppeteer. (n.d.). *Puppeteer documentation*. <https://pptr.dev/>
24. Mermaid. (n.d.). *Diagramming and charting tool*. <https://mermaid.js.org/>
25. docx. (n.d.). *Generate .docx files with JavaScript*. <https://docx.js.org/>
26. OpenJS Foundation. (n.d.). *ESLint: Find and fix problems in your JavaScript code*. <https://eslint.org/>
27. Prettier. (n.d.). *Prettier: An opinionated code formatter*. <https://prettier.io/>

References

28. Department of Trade and Industry. (n.d.). *MSME statistics*. <https://www.dti.gov.ph/resources/msme-statistics/>
29. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT) (RFC 7519)*. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc7519>
30. MDN Web Docs. (n.d.). *HTTP response status codes*. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
31. OWASP Foundation. (n.d.). *Password storage cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
32. MongoDB, Inc. (n.d.). *Atomicity and transactions*. <https://www.mongodb.com/docs/manual/core/write-operations-atomicity/>